

```

"use client";

import React, { forwardRef, useRef } from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface SwitchProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "type"
> {
  label?: string;
  error?: string;
  checked?: boolean;
  onChange?: (checked: boolean) => void;
}

export const Switch = forwardRef<HTMLInputElement, SwitchProps>(
  ({ label, error, checked, onChange, className, disabled, ...props }, ref) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const isChecked = checked ?? false;
    const hasError = !!error;
    const isActuallyDisabled = disabled ?? false;

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
      if (isActuallyDisabled) return;
      const newChecked = e.target.checked;
      onChange?.(newChecked);
    };

    const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
      if (e.key === " " || e.key === "Enter") {
        e.preventDefault();
        if (!isActuallyDisabled) {
          onChange?.(!isChecked);
        }
      }
    };

    const switchTrackClasses = cn(
      "relative inline-flex items-center",
      "w-11 h-6 shrink-0",
      "rounded-full border-2",
      "border-border bg-background",
      "transition-all duration-200 ease-out",
    );
  }
);

```

```

    "active:scale-95 duration-150 transition-all",
    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
    "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
    "aria-invalid:border-destructive aria-invalid:ring-3
aria-invalid:ring-destructive/20",
    isChecked
      ? "bg-brand-primary border-brand-primary shadow-sm hover:shadow-md"
      : "hover:border-brand-primary/50 hover:bg-brand-primary/5",
    hasError && "border-destructive focus-visible:ring-destructive",
    className,
  );

const thumbClasses = cn(
  "relative inline-block",
  "w-5 h-5 shrink-0",
  "rounded-full bg-white",
  "shadow-md shadow-black/10",
  "transition-transform duration-200 ease-spring",
  "active:scale-95",
  isChecked ? "translate-x-full" : "translate-x-0",
  hasError && "shadow-destructive/20",
);

const labelClasses = cn(
  "inline-flex items-center gap-2 cursor-pointer select-none",
  "transition-colors duration-200 ease-out",
  "text-text-main",
  hasError && "text-destructive/90",
  isActuallyDisabled && "opacity-50 pointer-events-none cursor-not-allowed",
);

return (
  <label className={labelClasses}>
    <input
      ref={el => {
        inputRef.current = el;
        if (ref) {
          if (typeof ref === "function") {
            ref(el);
          } else {
            ref.current = el;
          }
        }
      }}
      type="checkbox"
      role="switch"
      checked={isChecked}
      disabled={isActuallyDisabled}
    />
  </label>
);

```

```

    onChange={handleChange}
    onKeyDown={handleKeyDown}
    aria-checked={isChecked}
    aria-invalid={hasError ? "true" : "false"}
    aria-disabled={isActuallyDisabled}
    aria-required={props.required}
    className="sr-only peer"
    {...props}
  />
  <div className={switchTrackClasses} aria-hidden="true">
    <span className={thumbClasses} aria-hidden="true" />
  </div>
  {label && (
    <span
      className={cn(
        "text-sm font-medium leading-none",
        "transition-colors duration-200 ease-out",
        "text-text-main",
        hasError && "text-destructive/90",
        isActuallyDisabled && "opacity-50",
      )}
    >
      {label}
    </span>
  )}
</label>
);
},
);
Switch.displayName = "Switch";

```